Introduction to Robotics: ROS 2 Lab. 2

ROS 2 Introduction, Packages and Nodes

Professor: Pietro Falco
Tutor: Alessandro Adami, email: adamialess@dei.unipd.it

Department of Information Engineering (DEI)
Università degli studi di Padova

February 24, 2026

1. ROS 2 Introduction
2. ROS 2 Package
3. The ROS 2 Graph
4. ROS 2 Overview
5. The ROS 2 Multi-Terminal Workflow
6. ROS 2 TurtleSim
7. ROS 2 Nodes
8. VS Code Extensions
9. ROS 2 Communication
10. Lab 2: Hands-on Exercises

ROS 2 Introduction

The **Robot Operating System (ROS)** is not an actual operating system, but a flexible **middleware** framework for writing robot software.

- **The Goal**: To simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms.
- **Plumbing**: It provides message passing, package management, and hardware abstraction.
- **Tools**: Includes a powerful set of libraries (geometry, transforms) and visualizers (RViz, Gazebo).
- **Ecosystem**: A massive global community providing open-source drivers and algorithms.

Why ROS 2 over ROS1?

ROS 2 (specifically **Humble**) was designed for production-grade robotics, supporting multi-robot systems, real-time control, and improved security.

Why do we need ROS 2 instead of just writing standard Python scripts? ROS 2 acts as a **Middleware** layer that sits between your code and the OS.

- **Abstraction**: You don't need to know how the Lidar hardware works; you just listen to a `/scan` topic.
- **Language Independence**: A Python node can talk to a C++ node seamlessly.
- **Distributed Computing**: Nodes can run on different computers (e.g., a Raspberry Pi on the robot and a Laptop for visualization) and "find" each other over the network.

The "Standard" Language

ROS 2 provides a common language for robots. If you write a navigation algorithm for a TurtleBot, it should (theoretically) work on a 500kg industrial robot with minimal changes.

The core difference in ROS 2 is the use of **DDS**, an industry-standard communication protocol used in aerospace and defense.

- **No Master Node**: There is no single point of failure. If one node dies, the others continue communicating.
- **Discovery**: When you start a node, it "shouts" on the network: *"I am the Camera node, and I publish /image!"*. Other nodes listen and connect automatically.
- **Quality of Service (QoS)**: You can decide how data is handled.
  - **Reliable**: "Make sure every packet arrives" (best for commands).
  - **Best Effort**: "If a packet is lost, don't worry, just send the next one" (best for high-frequency sensor data).

ROS 2 Package

We will use ROS 2 functions to build a proper ROS 2 package.

```
ros2 pkg create $NAME --build-type ament_python --dependencies rclpy
# rclpy will include ROS 2 libs for Python
```

After that, you will find different files in your package:

- `package.xml`: It is present in any ROS 2 package and contains information like dependencies
- `setup.cfg`: **DON'T TOUCH IT! (unless instructed)** It helps during node installation
- `setup.py`: similar to `package.xml`
- test folder
- folder with the same name as the package: Here you will write your code

Then build the package:

```
colcon build
```

Now we have our Python-ROS 2 package, and we will use it for the following laboratories.

> **Colcon build**
>
> Always build from the root of the workspace (/ros2_ws).

A ROS 2 package is defined by two primary files:

- `package.xml`: The "Identity Card" of your project.
  - Contains the name, version, and maintainer info.
  - Defines **dependencies** (other ROS 2 packages your project needs).
- `setup.py`: The "Installation Manual."
  - Tells ROS 2 which Python files are executable nodes.
  - Defines the **entry points** (how you call your code from the terminal).

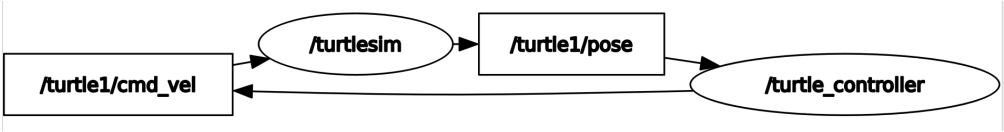
The ROS 2 Graph

ROS 2 processes are organized in a **Graph**. The main components are:

- **Nodes**: A single process that performs computation (e.g., one node for the Lidar driver, one for the motor controller).
- **Messages**: A standard data structure (e.g., an `Int32` or a `LaserScan`).
- **Topics**: The "bus" where nodes exchange messages.
- **Discovery**: Nodes automatically find each other on the network—no "Master" node required.

Modularity

If the "Camera Node" crashes, the "Navigation Node" can keep running. This modularity is why ROS 2 is robust.



How do we know if our Python nodes are actually talking to each other? We use **rqt_graph**, a GUI tool that visualizes the ROS 2 computation graph. A ROS 2 graph is a representation of all the nodes that are running.

- **Nodes**: Represented by ovals (your Python scripts, a Node can be seen as a running program with ROS 2 functionalities access).
- **Topics**: Represented by rectangles (the "pipes" carrying data).
- **Arrows**: Represent the flow of data (Publisher → Subscriber).

How to run it

```
# In a new terminal (we will see it later):
ros2 run rqt_graph rqt_graph
```

Debugging Tip

If your robot isn't moving, check `rqt_graph` first. If a node exists but has no arrows, your Topic names probably don't match!

Because Docker containers are "headless" (no screen), running `rqt_graph` requires a bridge to your Host OS display.

- **Linux:** Use `xhost +local:root` and map the `/tmp/.X11-unix` volume.
- **Windows/Mac:** You need an X-Server (like **VcXsrv** or **XQuartz**) installed on your host.

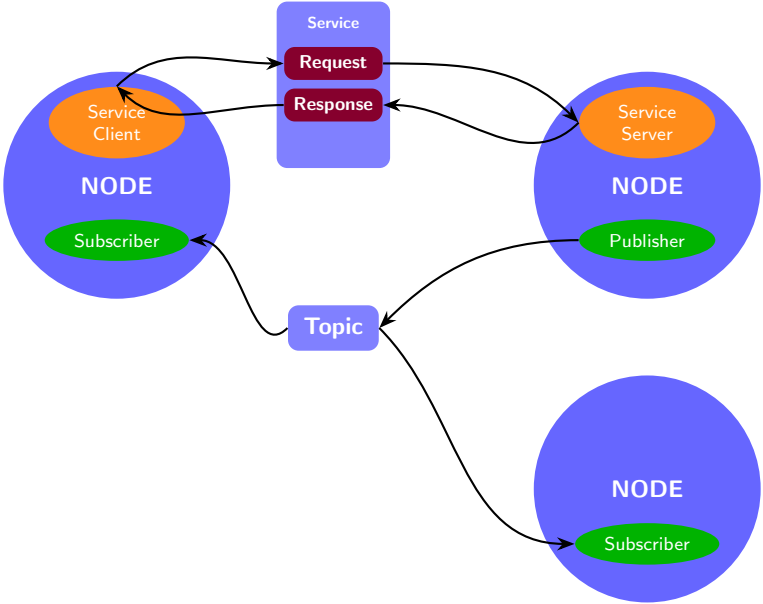
Docker Run Flags for GUI

Flag	Purpose
`-e DISPLAY=$DISPLAY`	Passes the display variable to the container.
`-v /tmp/.X11-unix:/tmp/.X11-unix`	Maps the graphical socket.

Alternative: VS Code Desktop

The VS Code "Remote Development" extension can sometimes forward ports, but for ROS 2 GUIs, the X11 bridge is the most robust method for the lab.

ROS 2 Overview


Service
Request
Response
Service
Client
NODE
Subscriber
Service
Server
NODE
Publisher
Topic
NODE
Subscriber

Different tasks require different communication styles.

Communication Comparison

Feature	Topics	Services
Pattern	Many-to-Many (Pub/Sub)	One-to-One (Req/Res)
Timing	Asynchronous (Stream)	Synchronous (Wait for reply)
Analogy	Radio Broadcast	Phone Call
Use Case	Sensor data, Odometry	Triggering a camera, Resetting a map

- **Publishers**: Send data to a topic.
- **Subscribers**: Receive data from a topic.

The ROS 2 Multi-Terminal Workflow

Unlike monolithic programs, ROS 2 applications consist of many independent nodes. To monitor and control them, you will need multiple active terminal sessions. For example:

- **Terminal 1 (The Core):** Running the main robot driver or simulator.
- **Terminal 2 (The Monitor):** Running `ros2 topic list` or `ros2 topic echo`.
- **Terminal 3 (The Controller):** Sending manual commands or running a teleop node.

Docker & Terminals

If you open a new terminal on your host machine, you are **outside** the container. To get a second terminal **inside** the container, you must run: `docker exec -it my_container_name bash`

First, restart the container in one terminal:

```
docker start -ai ros2_humble
```

Then, in a second terminal run:

```
docker exec -it ros2_humble bash
```

ROS 2 TurtleSim

Turtlesim is a tool for learning ROS 2 nodes, topics, and services.

1. **Install** (if not in Dockerfile):

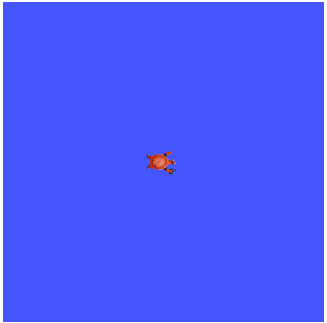
```
sudo apt update
sudo apt install ros-humble-turtlesim
```

2. **Run the Simulator**:

```
ros2 run turtlesim turtlesim_node
```

3. **Run the Keyboard Teleop** (In a new terminal):

```
ros2 run turtlesim draw_square
# OR for manual control:
ros2 run turtlesim turtle_teleop_key
#Click inside the terminal otherwise the turtle is not going to move
```



GUI Note

If you are on Windows/Mac, you may need an **X-Server** (like VcXsrv or XQuartz) to see the Turtle window!

While the turtle is running, use these commands to verify the underlying "plumbing."

- **Nodes**: See the active processes.

```
ros2 node list  # Should show /turtlesim
```

- **Topics**: See the data streams.

```
ros2 topic list # Look for /turtle1/cmd_vel
```

- **Echo**: Monitor real-time velocity data.

```
ros2 topic echo /turtle1/cmd_vel
```

Logic Check

If you see the nodes and topics, your environment is correctly configured for the upcoming labs!

Mastering the Command Line Interface (CLI) is the fastest way to debug your robot. All topic commands start with `ros2 topic`.

Essential Topic Commands

Command	Action	When to use it?
`list`	`ros2 topic list`	"What data streams are available?"
`echo`	`ros2 topic echo /topic`	"Show me the actual data in real-time."
`info`	`ros2 topic info /topic`	"Who is publishing and who is listening?"
`type`	`ros2 topic type /topic`	"What is the message structure (e.g. Twist)?"
`hz`	`ros2 topic hz /topic`	"Is the data coming in fast enough?"
`pub`	`ros2 topic pub /topic ...`	"Send a manual command to a node."

Example: Moving the Turtle manually

```
# Send a single command to move forward (linear x = 2.0)
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
{x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Tab Completion

Pro-Tip: You don't need to type everything! Type `ros2 topic pub /t` and hit Tab twice. ROS 2 will autocomplete the topic name and the message structure for you.

ROS 2 Nodes

In ROS 2, a **Node** is a process that performs computation. A single robotic system typically consists of many nodes coordinated together.

- **Single Responsibility**: Each node should do one thing (e.g., one for the Lidar, one for the wheel motors, one for path planning).
- **Modularity**: Nodes can be written in different languages (C++ or Python) and still talk to each other (we will see it later).
- **Fault Tolerance**: If a sensor node crashes, the rest of the system can remain active or enter a safe state.
- **Naming**: Every node must have a unique name within the system to be identified.

Philosophy

Think of a Node like a specialist in a team. You don't want one person doing everything; you want a team of experts communicating effectively.

In ROS 2 Humble, we use the `rclpy` library to create nodes. The standard practice is to use Object-Oriented Programming (OOP).
In your `your_pkg/your_pkg` create a new Python file

```
touch my_first_node.py
chmod +x my_first_node.py # make it executable
```

Then create and setup the node.
After you have written your node open `setup.py` file and add in `entry_points` the node as:

```
entry_points={
        'console_scripts': [ "test_node = ItR_lab_pkg.my_first_node:main"
        # node=pkg.file:main, main is the function that we want to run
        ],
    },
```

And then

```
colcon build --symlink-install
source install/setup.bash
```

Node Name

`test_node` is the **executable** name of the node and is used to run the node, you can also use the same name as the `.py` file.

--symlink-install

`--symlink-install` is a pro-tip for Python developers because it allows you to edit the `.py` file and run it immediately without rebuilding. You may eventually just use `colcon build` every time you make changes.

- `test_node`: The executable name you will type in the terminal: `ros2 run ItR_lab_pkg test_node`.
- `ItR_lab_pkg` : The folder containing your script.
- `my_first_node`: The name of your `.py` file.
- `main`: The specific function to execute.

Python3

You can run your code without making the node executable in setup.py, but it is not suggested. `~/ros2_ws/ItR_lab_pkg/ItR_lab_pkg python3 my_first_node.py`

If `ros2 node list` tells you a node is alive, `info` tells you what it is doing. It reveals the node's **Interface**.

Terminal Command

```
ros2 node info /turtlesim
```

What you will see:

- **Subscribers**: Topics the node is listening to (e.g., `/turtle1/cmd_vel` to receive movement commands).
- **Publishers**: Topics the node is broadcasting (e.g., `/turtle1/pose` to tell everyone its coordinates).
- **Service Servers**: Capabilities you can trigger (e.g., `/reset` to put the turtle back in the center).
- **Action Servers**: Complex tasks (e.g., `/turtle1/rotate_absolute`).

The Secret to Debugging

If Node A is supposed to talk to Node B, run `info` on both. Ensure Node A's **Publisher** topic name and type match Node B's **Subscriber** exactly.

VS Code Extensions

To set up your development environment, open the **Extensions** view in VS Code by clicking the square icon (■■) in the Activity Bar on the side or pressing Ctrl + ⇧ + X.

- Search for and install the following **Microsoft** extensions:

Recommended Extensions

Extension	Icon Description	Purpose
ROS	Blue dots square	ROS 2 tooling, build tasks, and URDF preview.
Python	Blue/Yellow Snakes	Required for `rclpy` code completion and linting.
Docker	Whale with Containers	To manage your ROS 2 containers easily.

How to Install:

1. Type the extension name in the search bar.
2. Verify the publisher is **Microsoft** (verified blue checkmark).
3. Click **Install**.

Lab Environment

If you are working inside a Docker container via the "Remote - Containers" extension, you may need to click **"Install in Container"** for these to work with your ROS 2 code.

A Minimal Node Template

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__("first_node")
        self.counter_=0
        self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        self.get_logger().info(f"Counter value: {self.counter_}")
        self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)
    node = MyNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Interpreter line:

```
#!/usr/bin/env python3
```

Import of Python libraries for ROS 2

```
import rclpy
from rclpy.node import Node
```

Create a new node, which inherits functions from the Node class with ROS 2 functionalities

```
class MyNode(Node):
```

Construct the node with name (in graph we will see that name)

```
def __init__(self):
    super().__init__("first_node")
```

Create a timer inside the node with a call back function. This is called every step by the function and keeps the node alive

```
    self.counter_=0
    self.create_timer(1.0, self.timer_callback) #try to change callback time
```

Function that "does something" at each step

```
def timer_callback(self):
    self.get_logger().info("Hello "+str(self.counter_))
    self.counter_ += 1
```

Create a main that can receive arguments

```
def main(args=None):
```

First of all initialize the ROS 2 communication and then create a node

```
    rclpy.init(args=args)
    node = MyNode()
```

Keep the node alive during the execution. It continues to run, press `Ctrl+c` to kill it

```
    rclpy.spin(node)
```

Shut down ROS 2 communication and destroy the node

```
    rclpy.shutdown()
```

Run the node

```
if __name__ == '__main__':
    main()
```

To write ROS 2 code in Python, we use the `rclpy` (ROS 2 Client Library for Python) package. It allows Python developers to access the core ROS 2 features.

- **The Wrapper**: Most of ROS 2's high-performance logic is written in C (the `rcl` layer). `rclpy` is a "wrapper" that makes these features available in Python.
- **Key Responsibilities**:
  - **Node Management**: Creating and destroying nodes.
  - **Communication**: Handling Publishers, Subscribers, Services, and Actions.
  - **Execution**: The **Executor** and **Spinning** (keeping the node alive to process data).
  - **Logging**: Providing the `get_logger()` interface for debugging.

Language Parity

Whether you use `rclcpp` (C++) or `rclpy` (Python), the behavior on the network is the same. You can have a Python Publisher talking to a C++ Subscriber without any extra configuration.

Every ROS 2 Python script follows a strict 4-step "ritual" managed by `rclpy`:

1. `rclpy.init()`: Initializes the communications for the specific process. *Must be called before creating a node.*
2. **Node Creation**: Instances your class (e.g., `node = MyNode()`).
3. `rclpy.spin(node)`: The "Infinite Loop." This tells Python to wait and listen for incoming messages or timer events. Without this, your script would finish in 0.1 seconds and close.
4. `rclpy.shutdown()`: Safely closes the connections and cleans up memory.

The Boilerplate Logic

```python
def main():
    rclpy.init()        # 1. Start
    node = MyNode()     # 2. Create
    rclpy.spin(node)    # 3. Wait/Execute
    rclpy.shutdown()    # 4. Clean up
```

In robotics, we rarely use `time.sleep(1.0)`. Instead, we use **Callbacks**.
A callback is a function that is "called back" by the system when a specific event occurs.

- **The Timer**: `self.create_timer(1.0, self.timer_callback)`
  - This tells ROS 2: "Every 1.0 seconds, please run my function."
- **Event-Driven**: The node doesn't sit idle; it can process sensor data or other events while waiting for the next timer "tick."
- **The Role of** `spin()`: Callbacks **only** run if the node is "spinning." `rclpy.spin()` is the engine that checks if it's time to run a callback.

The Logic

```python
def timer_callback(self):
    # This code executes automatically every 1s
    self.counter_ += 1
```

Why not sleep?

If you use `time.sleep(10)`, your node is "frozen" for 10 seconds and cannot receive emergency stop commands or sensor updates. **Never sleep in a ROS 2 callback!**

Instead of the standard `print()`, ROS 2 uses a sophisticated logging system.

- **Traceability**: Logs include the timestamp and the name of the node that sent the message.
- **Filtering**: You can choose the "severity" level of the message.

Logging Levels

Level	Command	Usage
INFO	`self.get_logger().info()`	General status (e.g., "Goal reached").
WARN	`self.get_logger().warn()`	Something is wrong, but I can continue.
ERROR	`self.get_logger().error()`	A major issue (e.g., "Lidar disconnected").
FATAL	`self.get_logger().fatal()`	System must stop immediately.

Example

```
self.get_logger().info(f"Battery at {self.soc}%")
```

Why use it?

ROS 2 logs are not just printed to the screen; they are also saved to `/.ros/log` and can be broadcast to other computers via the `/rosout` topic.

Once your nodes are running (like the `turtlesim_node`), use these commands to inspect them:

Node CLI Commands

Command	Result
`ros2 node list`	Lists all currently active nodes.
`ros2 node info /name`	Shows details: publishers, subscribers, services.
`ros2 run pkg_name node`	Launches a specific node from a package.

Debugging Tip

Always use `ros2 node info` first when your robot isn't moving. It will tell you if the node is actually "listening" to the correct topics.

A ROS 2 Workspace is a specific folder structure. If you build in the wrong place, your code won't run!

1. **The Root** (`/ros2_ws`): Where you run `colcon build`.
2. **The Source** (`/ros2_ws/src`): Where you put your packages.

The "Magic" Source Command

After building, ROS 2 doesn't know your package exists until you "source" the install folder:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash  # <--- CRITICAL STEP
```

Tip: If `ros2 run` can't find your package, 99% of the time it's because you forgot to `source install/setup.bash` in that specific terminal.

ROS 2 Communication

In ROS 2, we don't just "send data." We choose a communication pattern based on the robot's needs.

Communication Patterns

Pattern	Analogy	Best For...
Topics	A Radio Broadcast	Continuous data (Sensor streams, Odometry).
Services	A Phone Call	Quick Request/Response (Turning on a light).
Actions	A Goal-Oriented Task	Long-term goals with feedback (Navigating to a room).

- **Asynchronous (Topics):** Fire and forget. The sender doesn't wait for a reply.
- **Synchronous (Services):** The sender waits until the task is done.

Lab 2: Hands-on Exercises

Goal: Set up the environment where all your future code will live.

1. Create the workspace folders:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

2. Create a new Python package named as you prefer (e.g. `lab2_pkg`):

```
ros2 pkg create lab2_pkg --build-type ament_python
--dependencies rclpy geometry_msgs
```

3. Build the empty package to verify setup:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

Tip

The `geometry_msgs` dependency is crucial for Exercise 2, as it contains the `Twist` message type used to move robots.

Goal: Write a node that "monitors" the turtle and logs its status.

1. Inside `lab2_pkg/lab2_pkg/`, create a file named `turtle_monitor.py`.
2. Copy the "Minimal Node Template" from previous slides.
3. **Task:** Modify the `timer_callback` so it alternates between two messages: `"Turtle is Active"` and `"Monitoring...  "`.
4. Register the node in `setup.py`:

```
'console_scripts': [
    'monitor = lab2_pkg.turtle_monitor:main',
],
```

5. Build and Run:

```
ros2 run lab2_pkg monitor
```

Goal: Understand how the ROS 2 graph changes dynamically.

1. **Terminal 1**: Run `ros2 run turtlesim turtlesim_node`
2. **Terminal 2**: Run your new `monitor` node from Exercise 2.
3. **Terminal 3**: Run `rqt_graph`

Reflection Questions:

- Do you see the `/turtlesim` node and your `/first_node`?
- Why is there no arrow between them? (Hint: Are they sharing a Topic yet?)
- Use `ros2 node info /turtlesim` in a terminal. What topics is the turtle listening to?